

```

// api/stripe-webhook.js
//
// Listens for Stripe's "checkout.session.completed" event and automatically
// emails the order details to the drop shipper via Resend.
//
// SETUP REQUIRED (see notes at bottom of this file):
// 1. Add these environment variables in your Vercel project settings:
//    - STRIPE_SECRET_KEY          (already set up from checkout session code)
//    - STRIPE_WEBHOOK_SECRET      (get this after creating the webhook in Stripe)
//    - RESEND_API_KEY             (get this from resend.com)
// 2. Add a webhook endpoint in Stripe Dashboard pointing to:
//    https://yourdomain.com/api/stripe-webhook
//    Event to send: checkout.session.completed
// 3. npm install resend stripe (if not already installed)

import Stripe from 'stripe';
import { Resend } from 'resend';

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY);
const resend = new Resend(process.env.RESEND_API_KEY);

const DROP_SHIPPER_EMAIL = 'Woodlandtransfers@sasktel.net';

// Vercel needs the raw request body (unparsed) to verify the Stripe signature
export const config = {
  api: {
    bodyParser: false,
  },
};

function buffer(readable) {
  return new Promise((resolve, reject) => {
    const chunks = [];
    readable.on('data', (chunk) => chunks.push(chunk));
    readable.on('end', () => resolve(Buffer.concat(chunks)));
    readable.on('error', reject);
  });
}

export default async function handler(req, res) {
  if (req.method !== 'POST') {
    return res.status(405).send('Method Not Allowed');
  }
}

```

```

const sig = req.headers['stripe-signature'];
const rawBody = await buffer(req);

let event;
try {
  event = stripe.webhooks.constructEvent(
    rawBody,
    sig,
    process.env.STRIPE_WEBHOOK_SECRET
  );
} catch (err) {
  console.error('Webhook signature verification failed:', err.message);
  return res.status(400).send(`Webhook Error: ${err.message}`);
}

if (event.type === 'checkout.session.completed') {
  const session = event.data.object;

  try {
    // Pull full line item details (products, quantities) from Stripe
    const lineItems = await stripe.checkout.sessions.listLineItems(
      session.id,
      { expand: ['data.price.product'] }
    );

    const shipping = session.shipping_details || session.customer_details;
    const address = shipping?.address || {};

    const itemsHtml = lineItems.data
      .map((item) => {
        const name = item.description || item.price?.product?.name || 'Item';
        return `<li>${item.quantity} x ${name} - $$${(item.amount_total / 100).toFixed(2)}</li>`;
      })
      .join('');

    const emailHtml = `
      <h2>New Oil Ragzs Order</h2>
      <p><strong>Order/Session ID:</strong> ${session.id}</p>
      <p><strong>Customer:</strong> ${session.customer_details?.name || 'N/A'}</p>
      <p><strong>Customer Email:</strong> ${session.customer_details?.email || ''}</p>
      <h3>Items</h3>
      <ul>${itemsHtml}</ul>
      <h3>Shipping Address</h3>
      <p>
        ${shipping?.name || ''}<br/>
        ${address.line1 || ''} ${address.line2 || ''}<br/>
        ${address.city || ''}, ${address.state || ''} ${address.postal_code || ''}
      </p>
    `;
  }
}

```

```

    ${address.country || ''}
  </p>
  <p><strong>Total:</strong> ${((session.amount_total / 100).toFixed(2))} ${
`;

await resend.emails.send({
  from: 'Oil Ragzs Orders <orders@oilragzs.com>', // must be a verified dom
  to: DROP_SHIPPER_EMAIL,
  subject: `New Order - ${session.id}`,
  html: emailHtml,
});

console.log('Order email sent to drop shipper for session:', session.id);
} catch (err) {
  console.error('Error sending order email:', err);
  // Still return 200 so Stripe doesn't keep retrying indefinitely;
  // log/alert on this in production instead.
}
}

res.status(200).json({ received: true });
}

```